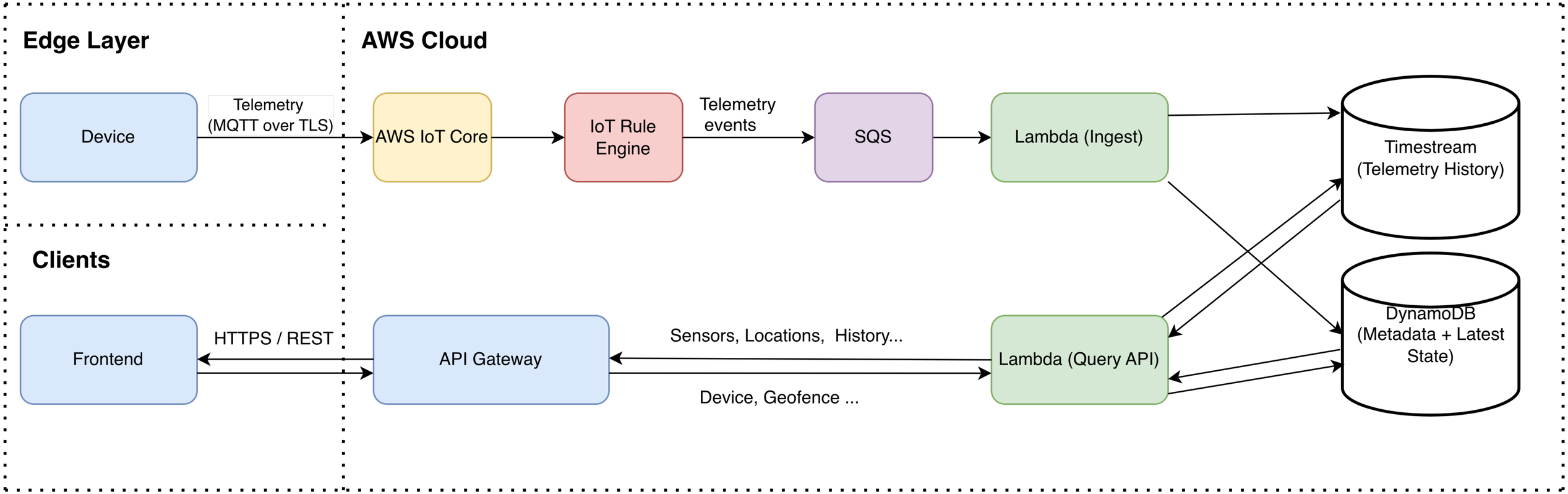
Fleet Management and Asset Tracking System

Eid Badr

Architecture



REST APIs and Testing

REST APIs

Method	Endpoint	Backend	Use Case
POST	`/devices`	DynamoDB	Register new device (idempotent)
GET	`/devices/{id}`	DynamoDB	Device info
GET	`/devices/{id}/locations`	Timestream	Route history (paginated)
GET	`/devices/{id}/latest-position`	DynamoDB	Real-time tracking (O(1) lookup)
GET	`/devices/{id}/sensors`	Timestream	Diagnostics

Testing

Layer	What	Tools
Unit	Validation, business logic	JUnit 5, Mockito
Handler	Lambda handler directly	JUnit 5 + API Gateway events
Integration	Lambda + AWS services	LocalStack, TestContainers
Contract	API matches OpenAPI spec	Atlassian Swagger Validator
E2E (Simulated)	Full pipeline with simulated devices	AWS IoT Device Simulator, Python scripts
E2E (Real Devices)	Physical devices in staging environment	Raspberry Pi / ESP32 test fleet

Design decisions and trade offs

Ingestion Architecture

Decision:

IoT Core → Rule Engine → SQS → Lambda

Why:

- decoupling & buffering
- handles burst traffic
- enables future fan-out (alerts, analytics)

Storage Strategy

Decision:

- Timestream → telemetry history
- DynamoDB → metadata + latest state

Why:

- optimized for time-series queries
- fast O(1) lookup for real-time tracking
- avoids heavy queries on Timestream

Scalability Strategy

- IoT Core handles millions of connections
- SQS buffers spikes
- Lambda scales horizontally
- DynamoDB & Timestream scale automatically

Storage Strategy

Decision:

- Timestream → telemetry history
- DynamoDB → metadata + latest state

Why:

- optimized for time-series queries
- fast O(1) lookup for real-time tracking
- avoids heavy queries on Timestream

Data Model

Decision: semi-structured

Why:

- supports vehicle + non-vehicle assets
- avoids schema explosion
- enables future device types

Security

- device auth via X.509 (IoT Core)
- API auth via JWT / Cognito (optional)
- IAM roles for service-to-service access